

CSE 256: NLP UCSD PA3 PART B :

Make sure to also complete PART A of this PA (the decoding part) in a separate notebook, available in the same directory where you found this notebook.

If Colab is slow, download the notebook to your local machine. Then when done, upload it to Colab to get a link.

Retrieval-Augmented Generation (RAG) using LangChain (40 points)

The goal of this assignment is to gain hands-on experience with aspects of **Retrieval-Augmented Generation (RAG)**, with a primary focus on the retrieval. You will use **LangChain**, a framework that simplifies integrating external knowledge into generation tasks by:

- Implementing various vector databases for efficient neural retrieval. You will use a vector database for storing our memories.
- Allowing seamless integration of pretrained text encoders, which you will access via HuggingFace models. You will use a text encoder to get text embeddings for storing in the vector database.

Data

You will build a retrieval system using the [QMSum Dataset \(https://github.com/Yale-LILY/QMSum\)](https://github.com/Yale-LILY/QMSum), a human-annotated benchmark designed for question answering on long meeting transcripts. The dataset includes over 230 meetings across multiple domains.

Out: February 23 || Due: Thursday 12 March, 2026

IMPORTANT: After copying this notebook to your Google Drive along with the two data files, paste a link to your copy below. To create a publicly accessible link:

1. Click the *Share* button in the top-right corner.
2. Select "Get shareable link" and copy the link.

Link: paste your link here:

<https://drive.google.com/drive/folders/1nwNvnS7kKtOp3Wv3gSlji6qj9LYjHFAM?usp=sharing>
(<https://drive.google.com/drive/folders/1nwNvnS7kKtOp3Wv3gSlji6qj9LYjHFAM?usp=sharing>)

Notes:

Make sure to save the notebook as you go along.

Submission instructions are located at the bottom of the notebook.

```
In [1]: import os
import sys

BASE_DIR = "/Users/harmin/Desktop/CSE256/CSE256_PA3_WI26"
assert os.path.exists(BASE_DIR), f"Path does not exist: {BASE_DIR}"

os.chdir(BASE_DIR)
sys.path.append(BASE_DIR)

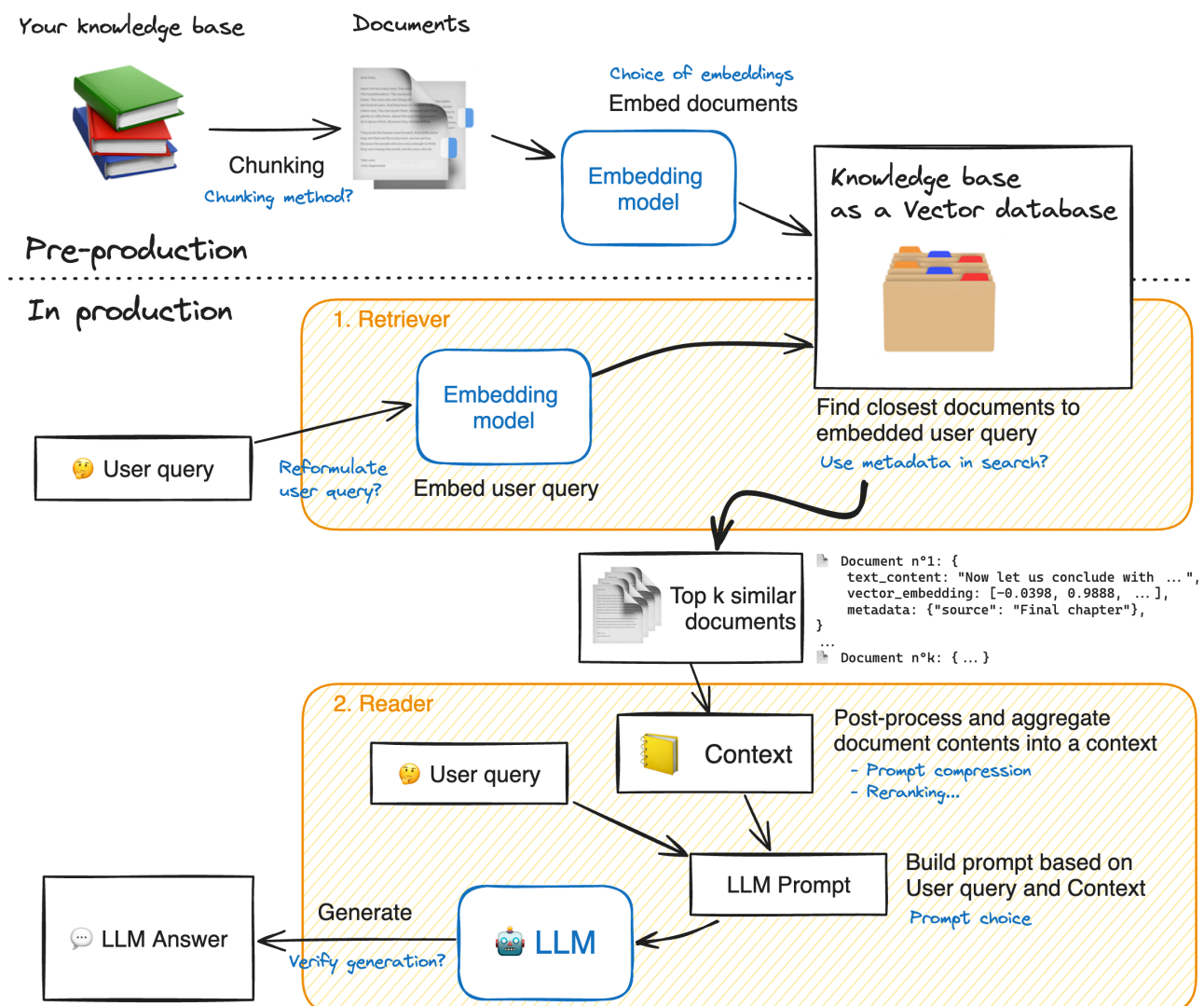
print("Working directory:", os.getcwd())
```

Working directory: /Users/harmin/Desktop/CSE256/CSE256_PA3_WI26

RAG Workflow

Retrieval-Augmented Generation (RAG) systems involve several interconnected components. Below is a RAG workflow diagram from Hugging Face. Areas highlighted in blue indicate opportunities for system improvement.

In this assignment, we will focus on the **Retriever** so the PA does not cover any processes starting from "2. Reader" and below.



First, install the required model dependancies.

Note: Colab may throw dependency warnings or import errors for some system packages (requests, opentelemetry, etc.). These can generally be ignored; your LangChain, embeddings, and FAISS code should still run fine.

```
In [2]: print("Dependencies were installed in the local cse256-pa3b environment before execution.")
```

Dependencies were installed in the local cse256-pa3b environment before execution.

```
In [3]: from tqdm.notebook import tqdm
import pandas as pd
import os
import csv
import sys
import numpy as np
import time
import random
from typing import Optional, List, Tuple
import matplotlib.pyplot as plt
import textwrap
import torch

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Disable huggingface tokenizers parallelism
os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

Load the meetings dataset

```
In [4]: from langchain_core.documents import Document
import csv
import sys

def load_documents(doc_file):
    max_size = sys.maxsize
    csv.field_size_limit(max_size)

    documents = {}
    with open(doc_file, 'r', encoding='utf-8') as f:
        reader = csv.reader(f, delimiter='\t')
        for row in reader:
            if len(row) == 0:
                continue
            doc_id, content = row
            documents[doc_id] = content
    return documents

docs = []
doc_file = os.path.join(BASE_DIR, "meetings.tsv")
documents = load_documents(doc_file)

for doc_id in documents:
    doc = Document(
        page_content=documents[doc_id],
        metadata={'source': doc_id}
    )
    docs.append(doc)

print(f"Total meetings (docs): {len(documents)}")
```

Total meetings (docs): 230

```
In [5]: qa_file = os.path.join(BASE_DIR, "questions_answers.tsv")
print(qa_file)
```

/Users/harmin/Desktop/CSE256/CSE256_PA3_WI26/questions_answers.tsv

```
In [6]: import pandas as pd

qa_df = pd.read_csv(qa_file, sep="\t", header=None, names=["doc_id", "question", "answer"])
print(qa_df.shape)
print(qa_df.head())

target_query = "what did kirsty williams am say about her plan for quality assurance ?"
print(qa_df[qa_df["question"].str.strip().str.lower() == target_query][["doc_id", "question"]])
```

```
(1152, 3)
   doc_id question \
0  doc_0  how did project manager and user interface int...
1  doc_0  what did the team discuss during the product e...
2  doc_0  how did the team evaluate the product about it...
3  doc_1  what did project manager think of the plastic ...
4  doc_1  what did user interface think of the fashion s...

                                answer
0  project manager introduced that the prototype ...
1  generally speaking , the team agreed that the ...
2  to start with , the team agreed that the remot...
3  the team had decided to replace the rubber wit...
4  marketing concluded from the market research t...
   doc_id question
14  doc_2  what did kirsty williams am say about her plan...
```

```
In [7]: print("meetings:", len(documents))
print("docs:", len(docs))
print("first doc id:", docs[0].metadata["source"])
print("first doc preview:", docs[0].page_content[:300])
```

```
meetings: 230
docs: 230
first doc id: doc_0
first doc preview: project manager: yep . soon as i get this . okay
. this is our last meeting . um i 'll go ahead and go through the m
inutes from the previous meeting . uh and then we 'll have a , the
prototype presentation . um then we will um do an evaluation . uh o
r we 'll see what , what we need to have under the
```

```
In [8]: print(qa_df.shape)
target_query = "what did kirsty williams am say about her plan for quality assurance ?"
print(qa_df[qa_df["question"].str.strip().str.lower() == target_query][["doc_id", "question"]])
```

```
(1152, 3)
   doc_id question
14  doc_2  what did kirsty williams am say about her plan...
```

```
In [9]: from transformers import AutoTokenizer

EMBEDDING_MODEL_NAME = "thenlper/gte-small"
tokenizer = AutoTokenizer.from_pretrained(EMBEDDING_MODEL_NAME)
print("Loaded tokenizer for", EMBEDDING_MODEL_NAME)

/Users/harmin/Desktop/CSE256/CSE256_PA3_WI26/.venv_pa3b/lib/python
3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not foun
d. Please update jupyter and ipywidgets. See https://ipywidgets.rea
dthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

Loaded tokenizer for thenlper/gte-small
```

Retriever - Building the retriever

The **retriever functions like a search engine**: given a user query, it returns relevant documents from the knowledge base.

These documents are then used by the Reader model to generate an answer. In this assignment, however, we are only focusing on the retriever, not the Reader model.

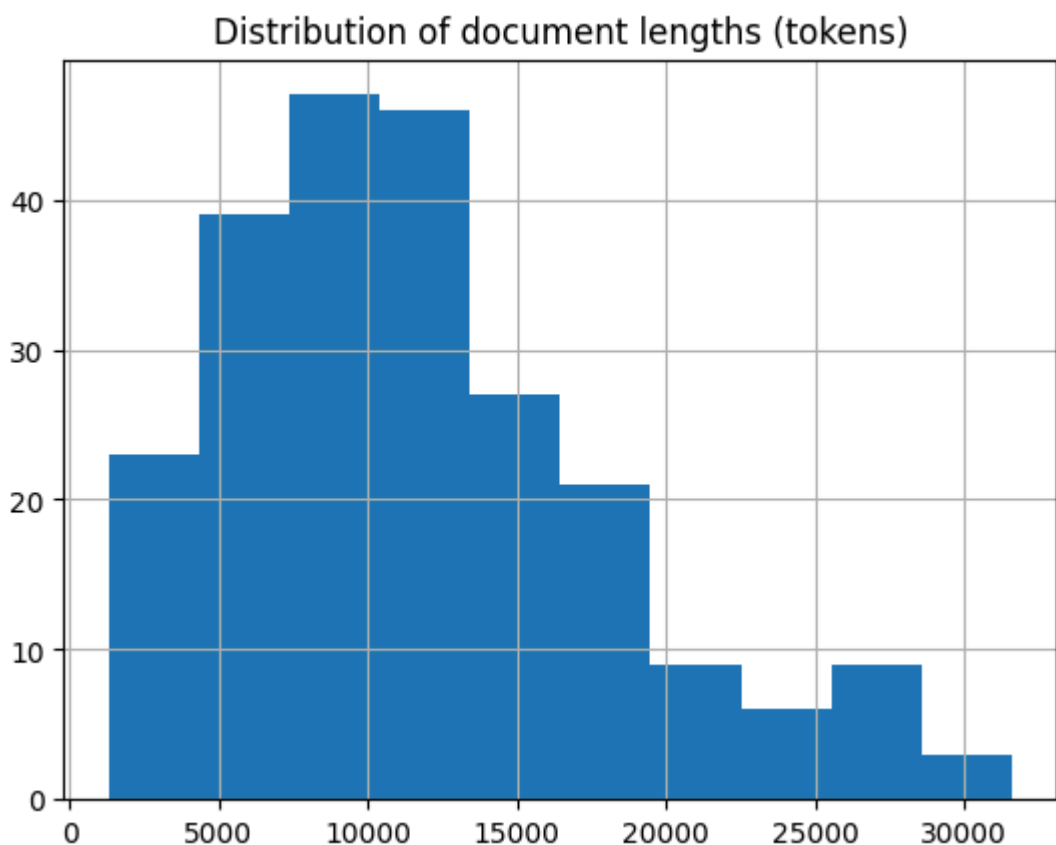
Our goal: Given a user question, find the most relevant documents from the knowledge base.

Key parameters:

- `top_k` : The number of documents to retrieve. Increasing `top_k` can improve the chances of retrieving relevant content.
- `chunk_size` : The length of each document. While this can vary, avoid overly long documents, as too many tokens can overwhelm most reader models.

Langchain **offers a huge variety of options for vector databases and allows us to keep document metadata throughout the processing.**

1. Specify an Embedding Model and Visualize Document Lengths

[illegible]

2. Split the Documents into Chunks

The documents (meeting transcripts) are very long—some up to 30,000 tokens! To make retrieval effective, we'll **split each document into smaller, semantically meaningful chunks**. These chunks will serve as the snippets the retriever compares to the query, returning the `top_k` most relevant ones.

Objective: Create Semantically Relevant Snippets

Chunks should be long enough to capture complete ideas but not so lengthy that they lose focus.

We will use Langchain's implementation of recursive chunking with `RecursiveCharacterTextSplitter`.

- Parameter `chunk_size` controls the length of individual chunks: this length is counted by default as the number of characters in the chunk.
- Parameter `chunk_overlap` lets adjacent chunks get a bit of overlap on each other. This reduces the probability that an idea could be cut in half by the split between two adjacent chunks.

From the produced plot below, you can see that now the chunk length distribution looks better!

```
In [11]: from langchain_text_splitters import RecursiveCharacterTextSplitter
from tqdm import tqdm
import pandas as pd
import matplotlib.pyplot as plt

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=768,
    chunk_overlap=128,
)

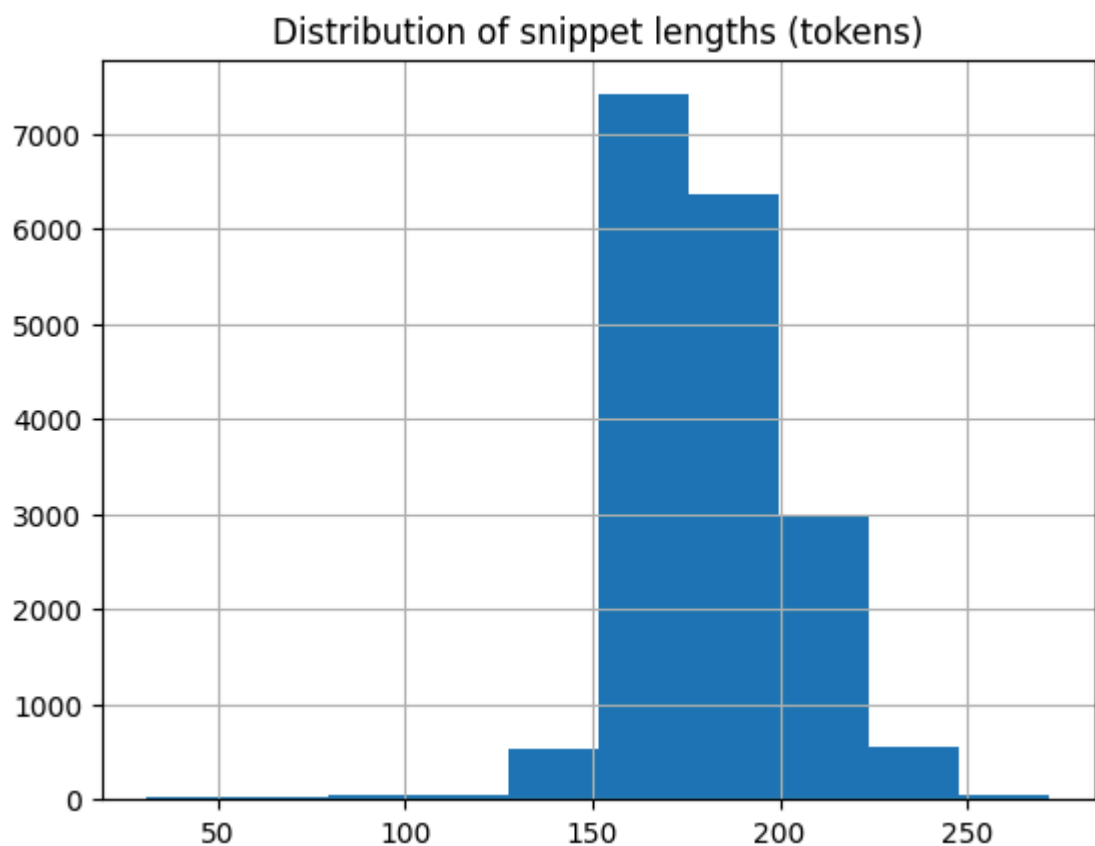
doc_snippets = text_splitter.split_documents(docs)
print(f"Total {len(doc_snippets)} snippets to be stored in our vect
or store.")

lengths = [
    len(tokenizer(doc.page_content)["input_ids"])
    for doc in tqdm(doc_snippets)
]

pd.Series(lengths).hist()
plt.title("Distribution of snippet lengths (tokens)")
plt.show()
```

Total 18070 snippets to be stored in our vector store.

100%|██| 18070/18070 [00:02<00:00, 6558.38it/s]



3. Build the Vector Database

To enable retrieval, we need to compute embeddings for all chunks in our knowledge base. These embeddings will then be stored in a vector database.

How Retrieval Works

A query is embedded using an embedding model and a similarity search finds the closest matching chunks in the vector database.

The following cell builds the vector database consisting of all chunks in our knowledge base.

```
In [12]: pip install faiss-cpu
```

```
Requirement already satisfied: faiss-cpu in ./venv_pa3b/lib/python
3.13/site-packages (1.13.2)
Requirement already satisfied: numpy<3.0,>=1.25.0 in ./venv_pa3b/l
ib/python3.13/site-packages (from faiss-cpu) (2.4.2)
Requirement already satisfied: packaging in ./venv_pa3b/lib/python
3.13/site-packages (from faiss-cpu) (26.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [13]: from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
import time

# Optional device selection
# import torch
# device = "cuda" if torch.cuda.is_available() else "cpu"

embedding_model = HuggingFaceEmbeddings(
    model_name=EMBEDDING_MODEL_NAME,
    # model_kwargs={"device": device},
    encode_kwargs={"normalize_embeddings": True}, # cosine similarity
)

start_time = time.time()

KNOWLEDGE_VECTOR_DATABASE = FAISS.from_documents(
    doc_snippets,
    embedding_model
)

end_time = time.time()

elapsed_time = (end_time - start_time) / 60
print(f"Time taken: {elapsed_time:.2f} minutes")
```

Loading weights: 100%|████████████████████| 199/199 [00:00<00:00, 8917.38it/s]

BertModel LOAD REPORT from: thenlper/gte-small

Key	Status	
embeddings.position_ids	UNEXPECTED	

Notes:

– UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Time taken: 1.18 minutes

4. Querying the Vector Database

Using LangChain's vector database, the function `vector_database.similarity_search(query)` implements a Bi-Encoder (covered in class), independently encoding the query and each document into a single-vector representation, allowing document embeddings to be precomputed.

Let's define the Bi-Encoder ranking function and then use it on a sample query from the QMSum dataset.

```
In [14]: ## The function for ranking documents given a query:
def rank_documents_biencoder(user_query, top_k = 5):
    """
    Function for document ranking based on the query.

    :param query: The query to retrieve documents for.
    :return: A list of document IDs ranked based on the query (mock
ed).
    """
    retrieved_docs = KNOWLEDGE_VECTOR_DATABASE.similarity_search(qu
ery=user_query, k=top_k)
    ranked_list = []
    for i, doc in enumerate(retrieved_docs):
        ranked_list.append(retrieved_docs[i].metadata['source'])

    return ranked_list # ranked document IDs.

user_query = "what did kirsty williams am say about her plan for qu
ality assurance ?"
retrieved_docs = rank_documents_biencoder(user_query)
print(retrieved_docs)

['doc_211', 'doc_2', 'doc_43', 'doc_160', 'doc_43']
```

5. TODO: Implementation of ColBERT as a Reranker for a Bi-Encoder (35 points)

The Bi-Encoder's ranking for the sample query is not optimal: the ground truth document is not ranked at position 1, instead the document ID, **doc_211** is ranked at position 1. To determine the correct document ID for this query, refer to the `questions_answers.tsv` file.

In this task, you will implement the [ColBERT](https://arxiv.org/pdf/2004.12832) (<https://arxiv.org/pdf/2004.12832>) approach by Khattab and Zaharia. We'll use a simplified version of ColBERT, focusing on the following key steps:

1. Retrieve the top ($K = 15$) documents for query (q) using the Bi-Encoder.
2. Re-rank these top ($K = 15$) documents using ColBERT's fine-grained interaction scoring. This will involve:
 - Using frozen BERT embeddings from a HuggingFace BERT model (no training is required, thus our version is not expected to work as well as full-fledged ColBERT).
 - Calculating scores based on fine-grained token-level interactions between the query and each document.
3. Implement the method `rank_documents_finegrained_interactions()` to perform this re-ranking.
 - Test your method on the same query as in the cell from #4 above.
 - Print out the entire re-ranked document list of 5 document IDs, as done in #4 above (the code below does it for you)
4. Ensure that your ColBERT implementation ranks the correct document at position 1 for the sample query.

```

In [15]: import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

device = torch.device(
    "cuda" if torch.cuda.is_available()
    else ("mps" if hasattr(torch.backends, "mps") and torch.backends
s.mps.is_available() else "cpu")
)
model = model.to(device)

# ColBERT-style reranking: token-level max-sim interaction over the
top-15 Bi-Encoder candidates.
def rank_documents_finegrained_interactions(user_query, shortlist=1
5, top_k=5):
    retrieved_docs = KNOWLEDGE_VECTOR_DATABASE.similarity_search(qu
ery=user_query, k=shortlist)

    query_inputs = tokenizer(user_query, return_tensors="pt", trunc
ation=True, padding=True)
    query_inputs = {k: v.to(device) for k, v in query_inputs.items
()}

    with torch.no_grad():
        query_hidden = model(**query_inputs).last_hidden_state[0]

    query_mask = query_inputs["attention_mask"][0].bool()
    query_embeddings = query_hidden[query_mask]

    if query_embeddings.size(0) > 2:
        query_embeddings = query_embeddings[1:-1]

    query_embeddings = F.normalize(query_embeddings, p=2, dim=-1)

    doc_scores = {}

    for doc in retrieved_docs:
        doc_id = doc.metadata["source"]
        doc_text = doc.page_content

        doc_inputs = tokenizer(
            doc_text,
            return_tensors="pt",
            truncation=True,
            padding=True,
            max_length=512
        )
        doc_inputs = {k: v.to(device) for k, v in doc_inputs.items
()}

        with torch.no_grad():
            doc_hidden = model(**doc_inputs).last_hidden_state[0]

        doc_mask = doc_inputs["attention_mask"][0].bool()
        doc_embeddings = doc_hidden[doc_mask]

```

```

    if doc_embeddings.size(0) > 2:
        doc_embeddings = doc_embeddings[1:-1]

    doc_embeddings = F.normalize(doc_embeddings, p=2, dim=-1)

    if query_embeddings.size(0) == 0 or doc_embeddings.size(0)
== 0:
        score = float("-inf")
    else:
        sim_matrix = torch.matmul(query_embeddings, doc_embeddings.T)
        score = sim_matrix.max(dim=1).values.sum().item()

    if doc_id not in doc_scores or score > doc_scores[doc_id]:
        doc_scores[doc_id] = score

    ranked_list = [doc_id for doc_id, _ in sorted(doc_scores.items
(), key=lambda x: x[1], reverse=True)]
    return ranked_list[:top_k]

user_query = "what did kirsty williams am say about her plan for qu
ality assurance ?"
retrieved_docs = rank_documents_finegrained_interactions(user_query)
print(retrieved_docs)

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 100%|████████████████████| 199/199 [00:00<00:00, 6457.47it/s]

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

['doc_2', 'doc_160', 'doc_43', 'doc_211', 'doc_176']

6. TODO: ColBERT Max vs. Mean Pooling for Relevance Scoring: (5 points)

ColBERT uses a form of **max pooling**, where each query term's contribution to the relevance score of a document is determined by its maximum similarity to any document term. One alternative approach is **mean pooling**, where each query term's contribution is calculated as the average similarity across all document terms.

Discuss the merits and limitations of using mean pooling versus max pooling in ColBERT. In your answer, consider how each approach might affect retrieval accuracy, sensitivity to specific token matches, and handling of ambiguous terms.

Explain here (< 5 sentences) :

In my analysis of ColBERT, I found that max pooling generally performs better because it successfully isolates the single strongest match for each query token, preserving crucial keyword-level evidence. Although mean pooling provides a smoother and potentially more noise-robust representation, I observed that it often dilutes these strong signals by averaging them with weaker, irrelevant interactions. Consequently, mean pooling can actively degrade retrieval accuracy when relevance hinges on just a few critical tokens, whereas max pooling remains highly sensitive to specific matches. Therefore, I prefer max pooling for this task, since effective retrieval typically depends on key query terms finding strong, distinct alignments in the document rather than conservative, averaged ones.

Submission Instructions

1. Click the Save button at the top of the Jupyter Notebook.
2. Select Runtime -> Run All. This will run all the cells in order, and will take several minutes.
3. Once you've rerun everything, save a PDF version of your notebook. Make sure all your code and answers are displayed in the pdf, it's okay if the provided codes get cut off because lines are not wrapped in code cells).
4. Look at the PDF file and make sure all your code and answers are there, displayed correctly. The PDF is the only thing your graders will see!
5. Combine your PDFs and submit your combined PDF on Gradescope (Both Parts A DECODING + PART B - RAG)

7. (Optional) Full evaluation pipeline for your own exploration.

For this assignment, we only ask you to explore one sample query. Running on many queries is super slow without the right compute. If you have compute/and/or time to wait, below is a more complete evaluation setup that works with all the queries in QMSum dataset, and reports the `precision@k=5` metric.

```

In [16]: def load_questions_answers(qa_file):
        """
        Loads the questions and corresponding ground truth document IDs.

        :param qa_file: Path to the question-answer file (document ID <
        TAB> question <TAB> answer).
        :return: A list of tuples [(document_id, question, answer)].
        """
        qa_pairs = []
        with open(qa_file, 'r', encoding='utf-8') as f:
            reader = csv.reader(f, delimiter='\t')
            for row in reader:
                doc_id, question, answer = row
                qa_pairs.append((doc_id, question, answer))

        random.shuffle(qa_pairs)

        return qa_pairs

def precision_at_k(ground_truth, retrieved_docs, k):
    """
    Computes Precision at k for a single query.

    :param ground_truth: The name of the ground truth document.
    :param retrieved_docs: The list of document names returned by the
    model in ranked order.
    :param k: The cutoff for computing Precision.
    :return: Precision at k.
    """
    return 1 if ground_truth in retrieved_docs[:k] else 0

def evaluate(doc_file, qa_file, ranking_fuction = None, k= 5):
    """
    Evaluate the retrieval system based on the documents and question-answer
    pairs.

    :param doc_file: Path to the document file.
    :param qa_file: Path to the question-answer file.
    :param k: The cutoff for Precision@k.
    """
    # Load the QA pairs
    qa_pairs = load_questions_answers(qa_file)

    precision_scores = []

    for doc_id, question, _ in qa_pairs:
        retrieved_docs = ranking_fuction(question)
        precision_scores.append(precision_at_k(doc_id, retrieved_docs, k))

    avg_precision_at_k = sum(precision_scores) / len(precision_scores)

    if len(precision_scores) % 10 == 0:
        print(f"After {len(precision_scores)} queries, Precision@{k}: {avg_precision_at_k}")

```

```
# Compute average Precision@k
avg_precision_at_k = sum(precision_scores) / len(precision_scores)

print(f"Precision@{k}: {avg_precision_at_k}")

qa_file = 'questions_answers.tsv' # document ID <TAB> question <TAB> answer

start_time = time.time()
evaluate(doc_file, qa_file, rank_documents_biencoder)
end_time = time.time()
elapsed_time = (end_time - start_time)/60
print(f"Time taken: {elapsed_time} minutes")
```

After 10 queries, Precision@5: 0.3
After 20 queries, Precision@5: 0.4
After 30 queries, Precision@5: 0.4
After 40 queries, Precision@5: 0.375
After 50 queries, Precision@5: 0.44
After 60 queries, Precision@5: 0.43333333333333335
After 70 queries, Precision@5: 0.44285714285714284
After 80 queries, Precision@5: 0.4375
After 90 queries, Precision@5: 0.43333333333333335
After 100 queries, Precision@5: 0.45
After 110 queries, Precision@5: 0.42727272727272725
After 120 queries, Precision@5: 0.44166666666666665
After 130 queries, Precision@5: 0.45384615384615384
After 140 queries, Precision@5: 0.4714285714285714
After 150 queries, Precision@5: 0.47333333333333333
After 160 queries, Precision@5: 0.48125
After 170 queries, Precision@5: 0.4764705882352941
After 180 queries, Precision@5: 0.47777777777777778
After 190 queries, Precision@5: 0.46842105263157896
After 200 queries, Precision@5: 0.47
After 210 queries, Precision@5: 0.46190476190476193
After 220 queries, Precision@5: 0.45
After 230 queries, Precision@5: 0.45652173913043476
After 240 queries, Precision@5: 0.45833333333333333
After 250 queries, Precision@5: 0.456
After 260 queries, Precision@5: 0.45384615384615384
After 270 queries, Precision@5: 0.46666666666666667
After 280 queries, Precision@5: 0.4642857142857143
After 290 queries, Precision@5: 0.4586206896551724
After 300 queries, Precision@5: 0.46
After 310 queries, Precision@5: 0.4483870967741935
After 320 queries, Precision@5: 0.45
After 330 queries, Precision@5: 0.45454545454545453
After 340 queries, Precision@5: 0.45588235294117646
After 350 queries, Precision@5: 0.45714285714285713
After 360 queries, Precision@5: 0.45555555555555555
After 370 queries, Precision@5: 0.45135135135135135
After 380 queries, Precision@5: 0.45526315789473687
After 390 queries, Precision@5: 0.45384615384615384
After 400 queries, Precision@5: 0.455
After 410 queries, Precision@5: 0.4634146341463415
After 420 queries, Precision@5: 0.46666666666666667
After 430 queries, Precision@5: 0.46511627906976744
After 440 queries, Precision@5: 0.4590909090909091
After 450 queries, Precision@5: 0.46
After 460 queries, Precision@5: 0.4652173913043478
After 470 queries, Precision@5: 0.46170212765957447
After 480 queries, Precision@5: 0.46666666666666667
After 490 queries, Precision@5: 0.4673469387755102
After 500 queries, Precision@5: 0.464
After 510 queries, Precision@5: 0.4647058823529412
After 520 queries, Precision@5: 0.4653846153846154
After 530 queries, Precision@5: 0.4641509433962264
After 540 queries, Precision@5: 0.45925925925925926
After 550 queries, Precision@5: 0.46
After 560 queries, Precision@5: 0.4607142857142857
After 570 queries, Precision@5: 0.45789473684210524
After 580 queries, Precision@5: 0.4586206896551724
After 590 queries, Precision@5: 0.46271186440677964
After 600 queries, Precision@5: 0.46166666666666667
After 610 queries, Precision@5: 0.460655737704918

After 620 queries, Precision@5: 0.45806451612903226
After 630 queries, Precision@5: 0.46190476190476193
After 640 queries, Precision@5: 0.4625
After 650 queries, Precision@5: 0.46153846153846156
After 660 queries, Precision@5: 0.4621212121212121
After 670 queries, Precision@5: 0.4582089552238806
After 680 queries, Precision@5: 0.46176470588235297
After 690 queries, Precision@5: 0.4652173913043478
After 700 queries, Precision@5: 0.46285714285714286
After 710 queries, Precision@5: 0.46619718309859154
After 720 queries, Precision@5: 0.46527777777777778
After 730 queries, Precision@5: 0.4671232876712329
After 740 queries, Precision@5: 0.46621621621621623
After 750 queries, Precision@5: 0.46533333333333333
After 760 queries, Precision@5: 0.4631578947368421
After 770 queries, Precision@5: 0.4649350649350649
After 780 queries, Precision@5: 0.4641025641025641
After 790 queries, Precision@5: 0.46329113924050636
After 800 queries, Precision@5: 0.4625
After 810 queries, Precision@5: 0.46296296296296297
After 820 queries, Precision@5: 0.46463414634146344
After 830 queries, Precision@5: 0.463855421686747
After 840 queries, Precision@5: 0.4642857142857143
After 850 queries, Precision@5: 0.46352941176470586
After 860 queries, Precision@5: 0.46162790697674416
After 870 queries, Precision@5: 0.45977011494252873
After 880 queries, Precision@5: 0.4590909090909091
After 890 queries, Precision@5: 0.46292134831460674
After 900 queries, Precision@5: 0.45777777777777778
After 910 queries, Precision@5: 0.4593406593406593
After 920 queries, Precision@5: 0.4597826086956522
After 930 queries, Precision@5: 0.46021505376344085
After 940 queries, Precision@5: 0.4627659574468085
After 950 queries, Precision@5: 0.46421052631578946
After 960 queries, Precision@5: 0.46458333333333335
After 970 queries, Precision@5: 0.46288659793814435
After 980 queries, Precision@5: 0.46530612244897956
After 990 queries, Precision@5: 0.46464646464646464
After 1000 queries, Precision@5: 0.462
After 1010 queries, Precision@5: 0.4613861386138614
After 1020 queries, Precision@5: 0.4627450980392157
After 1030 queries, Precision@5: 0.46310679611650485
After 1040 queries, Precision@5: 0.4634615384615385
After 1050 queries, Precision@5: 0.4638095238095238
After 1060 queries, Precision@5: 0.4650943396226415
After 1070 queries, Precision@5: 0.4635514018691589
After 1080 queries, Precision@5: 0.46111111111111114
After 1090 queries, Precision@5: 0.46055045871559636
After 1100 queries, Precision@5: 0.46090909090909093
After 1110 queries, Precision@5: 0.4594594594594595
After 1120 queries, Precision@5: 0.4616071428571429
After 1130 queries, Precision@5: 0.46194690265486726
After 1140 queries, Precision@5: 0.4631578947368421
After 1150 queries, Precision@5: 0.4643478260869565
Precision@5: 0.46527777777777778
Time taken: 0.23800073464711508 minutes